Network Intrusion Detection Using Machine Learning

A Reproducibility and Critical Evaluation Study Based on the NSL-KDD Dataset

Course: Data Science in Cyber

Submitted by: Nadine Shehab

ID : 212982912

Reference Paper: Nkiama, Said & Saidu (2016) — A Subset Feature Elimination Mechanism for Intrusion Detection System
Reference Implementation: Cynthia Koopman — Network-Intrusion-Detection (GitHub)

Table of Contents

Executive Summary

This report examines network intrusion detection on the NSL-KDD dataset, taking as its reference source the paper by Nkiama, Said, and Saidu (2016), "A Subset Feature Elimination Mechanism for Intrusion Detection System," together with the publicly available GitHub implementation produced by Cynthia Koopman that follows the same general approach. The purpose of the work is twofold: to reproduce a machine learning pipeline for classifying network connections as normal or malicious, and to critically examine the assumptions and claims associated with the reference methodology in light of the results actually obtained.

Two classifiers were trained for this study, a Decision Tree and a Random Forest, both built on a cleaned and reduced version of the NSL-KDD feature set after removing a constant column and six highly correlated attributes. On the designated NSL-KDD test partition, the Decision Tree achieved an accuracy of 79.09 percent, a precision of 96.51 percent, a recall of 65.64 percent, an F1-score of 78.14 percent, a Matthews Correlation Coefficient of 0.635, and a ROC-AUC of 0.813. The Random Forest achieved an accuracy of 77.52 percent, a precision of 96.66 percent, a recall of 62.67 percent, an F1-score of 76.04 percent, a Matthews Correlation Coefficient of 0.614, and a ROC-AUC of 0.958.

The most significant finding of this study concerns the sizeable gap between training performance and test performance. Five-fold cross-validation on the training partition produced accuracy figures of 99.81 percent for the Decision Tree and 99.89 percent for the Random Forest, yet both models fall to below 80 percent accuracy once evaluated on the held-out test set. Investigation of this gap traced it to seventeen attack types, comprising 3,750 test records, that appear in the NSL-KDD test partition but were never present during training. This discrepancy is used throughout the report as the central evidence for questioning how much confidence should be placed in high training or cross-validation accuracy figures reported for feature-selection and classification methods applied to this dataset, including the claims made in the reference paper.

Beyond this central argument, the report also documents the feature engineering decisions applied in this project, summarises the exploratory data analysis that motivated them, assesses the reproducibility of the reference source, presents the full experimental results with supporting confusion matrices, analyses the errors made by each model with particular attention to the trade-off between false positives and false negatives, and concludes with an assessment of the strengths and limitations of the original methodology together with suggestions for improvement.

Summary of the Source

Before the reference source can be evaluated, it is necessary to summarise what it contains and why it was chosen as the basis for this project. The following subsections first set out the rationale behind the selection of the paper, the repository, and the dataset, and then describe the problem the reference paper addresses, why that problem matters, the solution the authors propose, the dataset on which their work is built, and the methodology they follow.

2.1 Source Selection Rationale

Intrusion Detection Systems occupy a central place in modern cybersecurity because perimeter defences such as firewalls and access controls are no longer sufficient on their own; attackers routinely bypass these defences or exploit already-trusted access. IDS technology addresses this gap by continuously monitoring traffic for behaviour that departs from what is expected, and the growing sophistication of attacks has driven a shift away from static, signature-based detection towards data-driven, machine learning-based approaches capable of learning the statistical structure of normal and malicious traffic rather than relying on fixed rules.

The NSL-KDD dataset was selected as the empirical basis for this project because it remains one of the most widely used benchmarks in intrusion detection research. It was constructed specifically to correct structural weaknesses in the earlier KDD Cup 1999 dataset, most notably a very large volume of duplicate records that biased earlier models towards frequent classes and produced misleadingly optimistic results. Because the reference paper itself uses NSL-KDD, adopting the same dataset here allows the results obtained in this project to be meaningfully related to the published literature rather than existing in isolation.

The GitHub repository developed by Cynthia Koopman was selected as the technical reference because it provides a complete, openly accessible implementation built specifically around NSL-KDD, organised so that data preparation, feature handling, and model construction can each be examined individually. Critically, the repository was built with direct reference to Nkiama, Said, and Saidu (2016), which means it is not an isolated coding exercise but a concrete attempt to translate a published feature elimination methodology into working code. This link between a peer-reviewed paper and a runnable implementation is precisely what makes the source suitable for the reproducibility and critical evaluation aims of this project, which extend beyond simply reproducing the original results to questioning the assumptions, experimental design, feature engineering choices, and reported outcomes of the reference work.

2.2 Problem Being Addressed

The reference paper is concerned with the difficulty that arises once machine learning is applied to network intrusion detection at scale. Network connections in datasets such as NSL-KDD are described using a large number of attributes covering connection duration, protocol type, byte counts, error rates, and statistical summaries of recent connections, among others. Not every attribute contributes equally to distinguishing normal from malicious traffic, and some are redundant or only weakly related to the attack classes. When such attributes are retained without scrutiny, the resulting detection model becomes slower to train, harder to interpret, and in some cases less accurate. The central problem the authors tackle is therefore one of dimensionality: given a dataset with a substantial number of connection features, how can the subset that actually contributes to accurate classification be identified and separated from attributes that add complexity without adding value.

2.3 Importance of the Problem in Cybersecurity

This problem matters because intrusion detection systems must process continuous streams of traffic with minimal delay to be operationally useful. A detection model built on an unnecessarily large feature set requires more time and computing resources to evaluate each connection, which can translate directly into a slower response to genuine threats in environments where attacks unfold within seconds. Beyond speed, irrelevant or noisy features can degrade detection accuracy itself: a classifier trained on attributes that do not meaningfully differ between normal and malicious connections may learn spurious associations that fail to generalise, producing false alarms or, more seriously, missed detections once deployed against real traffic. Feature reduction is therefore not a purely academic exercise but a matter of direct practical consequence for the responsiveness and reliability of automated detection.

2.4 Proposed Solution

To address this problem, the authors propose a feature elimination mechanism that relies on a tree-based classifier to assign each feature a measure of importance, reflecting how strongly it contributes to separating traffic classes when the classifier is trained. Features that contribute little to this separation are treated as candidates for removal, while those with a consistently stronger relationship to the class label are retained. This is applied recursively: the classifier is trained, importance scores are examined, the weakest features are removed, and the classifier is retrained on the reduced set, progressively narrowing the full feature space down to a smaller subset that the authors argue preserves, and in some respects improves, classification performance relative to the full feature set.

2.5 Dataset Used

Both the reference paper and the associated repository build their experiments around NSL-KDD. Each record corresponds to a single network connection described by a fixed set of basic, content-based, and traffic-based attributes, together with a label identifying the connection as normal or as belonging to one of several attack categories such as denial-of-service, probing, or unauthorised access attempts. The dataset is provided as separate training and testing partitions, and, as demonstrated later in this report, the testing partition deliberately includes attack patterns not present in training, which places a genuine demand on the generalisation ability of any model evaluated on it.

2.6 Methodology Employed

The methodology followed in the reference paper begins with preparing the NSL-KDD data into a form suitable for a tree-based learning algorithm, converting categorical attributes into a numerical representation and organising the data into the supplied training and testing partitions. A tree-based classifier is trained on the full set of available features, and the importance the classifier assigns to each feature is recorded. Using these scores, the least contributing features are progressively removed and the classifier retrained at each stage, forming the recursive elimination cycle at the core of the proposed method. The GitHub repository mirrors this general procedure in code, implementing the data preparation steps, the feature importance ranking, and the recursive reduction cycle in a form that can be executed directly on NSL-KDD.

Critical Evaluation of the Author's Claims

Rather than relying only on reported accuracy, this section evaluates whether the claims made in the reference paper are supported by the evidence produced during this reproduction study.

3.1 What Did the Original Paper Claim?

The paper argues that its subset feature elimination mechanism removes redundant features while maintaining or improving intrusion detection performance. It also suggests that the reduced feature set produces an effective and efficient classifier on the NSL-KDD dataset.

3.2 Did the Experimental Results Support These Claims?

The results provide partial support. Feature reduction successfully produced strong models, but the reproduced results showed a large difference between training and independent testing performance. Five-fold cross-validation reached approximately 99.8% accuracy for both Decision Tree and Random Forest, whereas test accuracy fell to 79.09% and 77.52%, respectively. This indicates that excellent training performance does not necessarily translate into strong generalisation.

3.3 Was the Original Method Reproducible?

The overall pipeline was reproducible because the dataset, software libraries and implementation were publicly available. The preprocessing steps and model training could be repeated successfully. Minor implementation choices, such as encoding and software versions, may slightly affect results but do not prevent reproduction.

3.4 Why Were the Results Different?

The main reason was the structure of the NSL-KDD dataset itself. The test partition contains 17 attack types that never appear in the training partition, affecting approximately 3,750 test records. Since supervised models cannot learn classes that never appear during training, these unseen attacks account for most misclassifications, especially within the R2L category. Although Random Forest achieved a slightly higher cross-validation score, the Decision Tree produced slightly better independent test accuracy, suggesting that the ensemble did not generalise better under these conditions.

3.5 What Are the Limitations of the Original Approach?

The approach relies entirely on supervised learning using historical attack data. Consequently, it struggles with previously unseen attacks and with minority classes such as R2L and U2R. High cross-validation accuracy can therefore give an overly optimistic impression of real detection capability if independent testing is not emphasised.

3.6 Are the Paper's Conclusions Justified?

The evidence suggests that the paper's conclusions are justified only in part. The feature elimination strategy is effective for reducing redundant attributes and building accurate classifiers on known attack patterns. However, the reproduced experiments show that claims about overall detection performance should be interpreted cautiously because generalisation to unseen attacks remains limited. The independent test results therefore provide a more realistic assessment than training accuracy alone.

Feature Engineering Analysis

The feature engineering carried out for this project followed three main steps: encoding the categorical attributes, scaling the numeric attributes, and removing a set of highly redundant features identified through correlation analysis. Each of these steps carries both a technical and a cybersecurity-specific rationale.

4.1 Encoding of Categorical Attributes

NSL-KDD contains three categorical attributes describing each connection: protocol_type, with three possible values (tcp, udp, icmp); service, with seventy possible values describing the network service involved; and flag, with eleven possible values describing the status of the connection. These attributes were converted into numeric form using a LabelEncoder fitted on the training partition, with categories in the test partition that were never seen during training mapped to a sentinel value of minus one rather than allowed to raise an error.

This approach is practical and keeps the pipeline running end to end, but it carries a known limitation. LabelEncoder assigns an arbitrary integer to each category, which imposes an ordinal structure, and by extension a notion of numerical distance, on attributes that are purely nominal. There is no meaningful sense in which one service or flag value is numerically closer to another, yet a tree-based model or any distance-sensitive component of the pipeline may still be influenced by the ordering that the encoder happens to produce. For the Decision Tree and Random

Forest models used here, this risk is comparatively contained because tree-based splits partition on threshold values without needing the encoded values to reflect true distances, but the practice remains a simplification that a more refined pipeline might replace with a scheme that does not impose spurious order, such as target encoding or one-hot encoding restricted to the most frequent categories. The sentinel value of minus one used for unseen test-time categories is a further point worth flagging: because this value falls outside the ordinary encoded range, a tree-based model can, in principle, learn a specific rule for it, but this treats a structurally different situation, namely a category never seen in training, as if it were simply another observed value, which is a pragmatic compromise rather than a principled solution to the unseen-category problem.

4.2 Standardisation of Numeric Attributes

All numeric features were standardised using a StandardScaler fitted on the training partition and subsequently applied to the test partition, ensuring that no information from the test set influenced the scaling parameters. Standardisation is a common and generally sound practice given the very different scales present in NSL-KDD, where attributes such as src_bytes and dst_bytes can range into the hundreds of millions while rate-based attributes such as same_srv_rate are bounded between zero and one.

It is worth noting, however, that Decision Tree and Random Forest classifiers evaluate splits based on threshold comparisons within a single feature at a time, and are therefore invariant to monotonic rescaling of that feature. Standardisation consequently has no effect on the splits these particular models learn, or on their resulting predictions. Its inclusion in the pipeline is not harmful, and it would matter a great deal if the pipeline were later extended to include distance-based or gradient-based models such as k-nearest neighbours, support vector machines, or neural networks, but for the two tree-based algorithms actually used in this project it represents a step that adds processing overhead without changing the outcome, and its main value here is in keeping the pipeline consistent should other model families be introduced later.

4.3 Removal of Highly Correlated Features

Correlation analysis conducted on the training data identified several groups of features with a Pearson correlation coefficient above 0.9, indicating near-linear redundancy. On this basis, six features were removed from the modelling set: num_root, srv_serror_rate, dst_host_srv_serror_rate, dst_host_serror_rate, srv_rerror_rate, and dst_host_srv_rerror_rate, reducing the working feature set from forty-one columns (after the earlier removal of the constant column num_outbound_cmds) to thirty-five.

The correlation between num_root and num_compromised was 0.999, indicating that the two attributes are, for practical purposes, measuring the same underlying event. Both attributes increase in tandem during connections where a user-to-root style compromise has occurred and the attacker gains elevated access, so num_root adds negligible additional information once num_compromised is retained, while its removal reduces the risk of a tree-based model splitting importance across two attributes that encode essentially one signal.

The serror_rate family shows a comparable pattern: srv_serror_rate correlates with serror_rate at 0.993, and both dst_host_serror_rate and dst_host_srv_serror_rate correlate with serror_rate and with each other at levels between 0.977 and 0.986. These attributes all describe the frequency of SYN-type connection errors, measured at the level of the individual connection, the service, and the destination host, and they tend to rise together because a SYN-flood-style denial-of-service attack produces elevated error rates across all of these levels of aggregation simultaneously. Retaining serror_rate alone preserves the discriminative signal associated with this attack pattern while removing three attributes that add dimensionality without adding independent information. The rerror_rate family, comprising rerror_rate, srv_rerror_rate, dst_host_srv_rerror_rate, and dst_host_rerror_rate, shows the same structural redundancy for reset-type connection errors and was reduced in the same manner, retaining rerror_rate as the representative attribute.

From a purely mathematical standpoint, retaining a set of near-collinear features does not necessarily change what a tree-based ensemble is theoretically capable of learning, since a single feature from a correlated group can typically substitute for the others in any given split. In practice, however, near-collinearity dilutes the feature importance scores computed by these models across multiple redundant attributes rather than concentrating them on one representative attribute, which complicates the interpretation of feature importance and can marginally slow training without a corresponding gain in accuracy. Removing the redundant members of each correlated group therefore

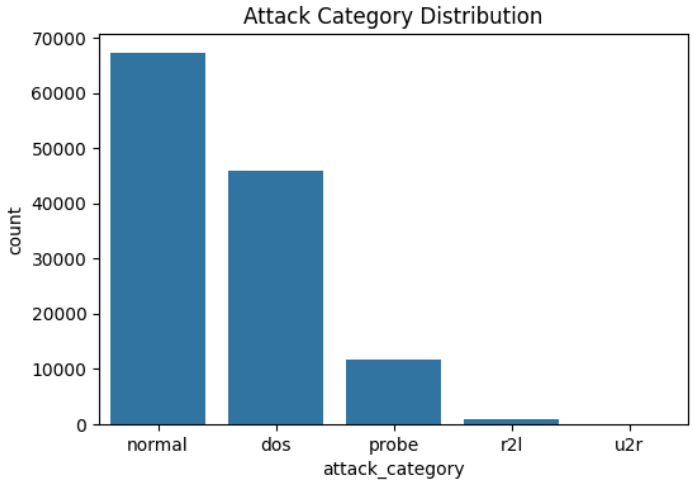
produces a leaner, more interpretable feature set aligned with the same feature-reduction philosophy that motivates the reference paper's own methodology.

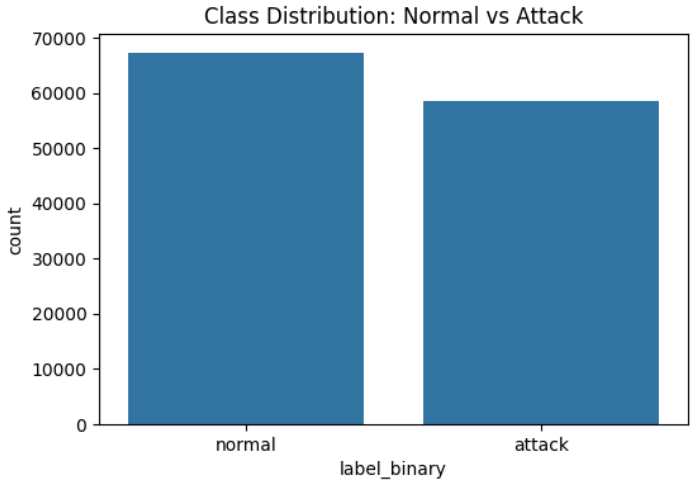
Exploratory Data Analysis Summary

Exploratory analysis of the training data examined class balance, feature correlation, and the presence of extreme values across key numeric attributes, and informed the feature engineering decisions described in the previous section.

5.1 Class Imbalance

Grouping the twenty-three original NSL-KDD labels into the standard five attack categories shows a training partition composed of 53.46 percent normal traffic, 36.46 percent denial-of-service traffic, 9.25 percent probing traffic, 0.79 percent remote-to-local traffic, and only 0.04 percent user-to-root traffic.

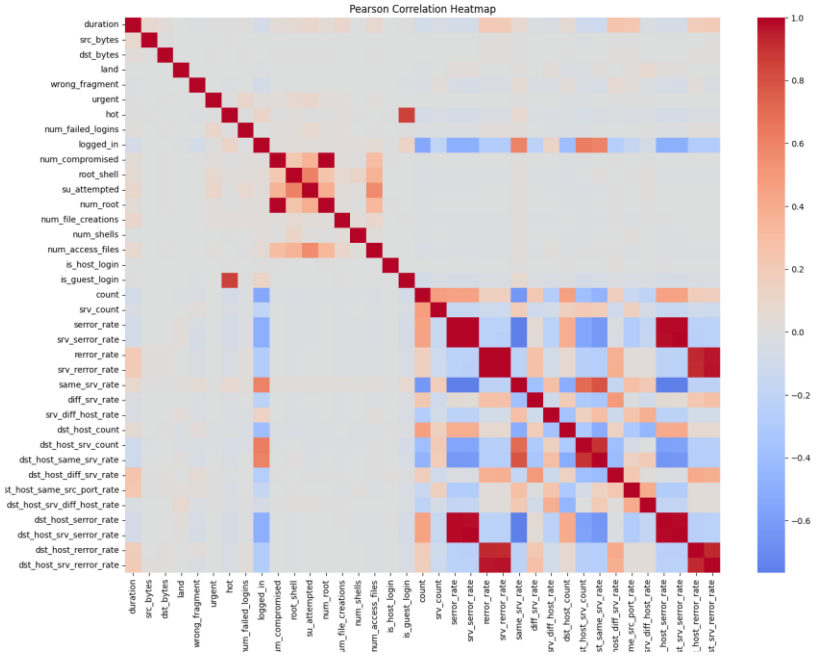




This imbalance reflects a realistic characteristic of network traffic rather than an artefact of data collection. Denial-of-service attacks such as neptune and smurf generate a very large number of connection records in a short period, since the attack itself consists of flooding a target with traffic, so they are naturally over-represented relative to the number of distinct attack episodes they correspond to. Remote-to-local and user-to-root attacks, by contrast, typically involve a small number of carefully targeted connections aimed at gaining unauthorised access or elevated privileges, and are consequently rare in the data despite often being more severe in their real-world consequences, since they can lead to credential theft or full system compromise rather than temporary service disruption. This

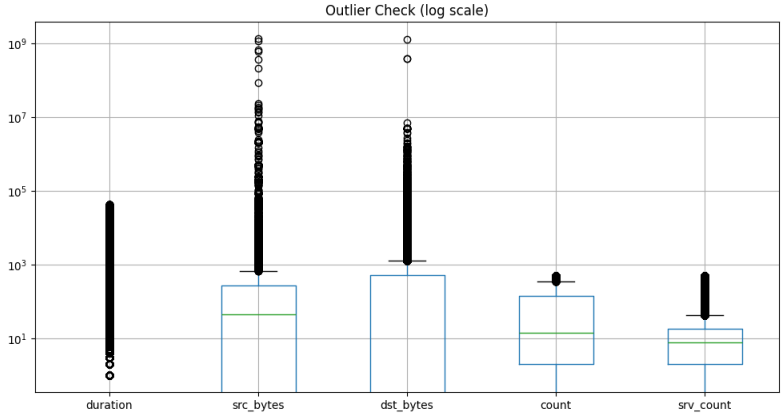
severity-versus-frequency mismatch has a direct bearing on the modelling results reported later in this document: with only 0.79 percent and 0.04 percent of training records belonging to the r2l and u2r categories respectively, both classifiers had very limited exposure to these attack patterns during training, which is consistent with r2l appearing as one of the two largest sources of classification error on the test set.

5.2 Correlation Structure



Pearson correlation was used to examine relationships between the numeric attributes because the great majority of NSL-KDD features are continuous count or rate variables, such as byte counts, connection counts, and error rates bounded between zero and one, for which a linear measure of association is an appropriate and standard default. The heatmap above highlights two visible blocks of strong positive correlation, corresponding to the serror_rate family and the rerror_rate family discussed in the feature engineering section, along with the near-perfect correlation between num_root and num_compromised. These groupings, together with the numeric correlation table produced during analysis, provided the direct evidence used to select the six features removed prior to modelling.

5.3 Outlier Behaviour



A log-scale boxplot of duration, src_bytes, dst_bytes, count, and srv_count shows pronounced right-skew and a large number of extreme values, particularly for src_bytes and dst_bytes, which range up to roughly a billion in the most extreme records. In many analytical contexts such extreme values would be treated as candidates for removal or capping, but in an intrusion detection setting this treatment would be inappropriate: unusually large byte counts, connection durations, or connection counts are frequently the very signature that separates an attack, such as a large data exfiltration attempt or a flooding attack, from ordinary traffic. Removing these records would risk discarding genuine positive examples of the attack behaviour the classifiers are meant to learn, so no outlier removal was applied to these attributes, and the log-scale transformation was used purely as a visualisation aid to make the distribution inspectable rather than as a precursor to filtering.

Reproducibility Analysis

An assessment of reproducibility was carried out by executing the full pipeline described in this report end to end, from data loading through to model evaluation, and by considering the dependencies, configuration choices, and any implicit steps that could affect whether an independent party would obtain comparable results.

6.1 Executability and Dependencies

The pipeline runs to completion using a standard Python data science stack comprising pandas, numpy, matplotlib, seaborn, and scikit-learn, all of which are freely available through the standard package index and are already present in most data science environments. No proprietary libraries, paid services, or non-public resources are required at any stage. The NSL-KDD dataset itself is publicly downloadable in its standard KDDTrain+ and KDDTest+ form, which removes any barrier to independently repeating the data preparation and modelling steps described here.

6.2 Implicit Steps and Configuration Sensitivity

A small number of implementation details affect exact reproducibility even though they do not prevent it. The LabelEncoder used for protocol_type, service, and flag assigns integer codes based on the alphabetical ordering of the categories observed in the training data; if the training data used to fit the encoder changes even slightly, or if a different NSL-KDD file arrangement is used, the resulting integer codes could shift, which would not change the models' effective behaviour but would change the specific encoded values seen in intermediate inspection. Both classifiers were instantiated with a fixed random_state of 42, and the Random Forest additionally fixes n_estimators at 100, which ensures that results obtained by rerunning this exact code will match to the reported precision. Scikit-learn's default hyperparameters for both classifiers have been stable across recent library versions, but a reproduction attempt using a substantially older or newer version of the library should confirm that defaults have not changed for the specific classifier settings used here.

6.3 Overall Assessment

Taken as a whole, the pipeline exhibits a high degree of reproducibility. The data source is public, the dependencies are standard and freely available, the random elements of model training are seeded, and the sequence of preprocessing steps, including the encoding scheme, the scaler, and the specific correlated features removed, is fully documented in the accompanying notebook. The main caveats relate to the sensitivity of encoded category values to the exact composition of the training data and to library-version-dependent defaults, both of which are common and well-understood limitations rather than obstacles specific to this project.

Experimental Results

This section presents the full set of results obtained from the Decision Tree and Random Forest classifiers, evaluated on the designated NSL-KDD test partition (KDDTest+), consisting of 22,544 records of which 9,711 are labelled normal and 12,833 are labelled as some form of attack.

7.1 Cross-Validation Results

Five-fold cross-validation on the training partition, prior to any evaluation on the held-out test set, produced the following accuracy figures:

Model	5-Fold Cross-Validation Accuracy (Training Set)
Decision Tree	99.81%
Random Forest	99.89%

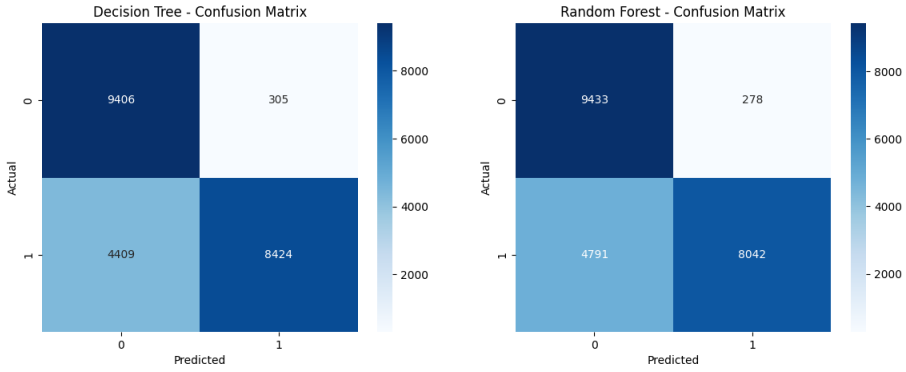
7.2 Test-Set Performance

Evaluation of both trained models on the independent test partition produced the following metrics:

Model	Accuracy	Precision	Recall	F1-score	MCC	ROC-AUC
Decision Tree	79.09%	96.51%	65.64%	78.14%	0.635	0.813
Random Forest	77.52%	96.66%	62.67%	76.04%	0.614	0.958

The gap between the cross-validation figures in Section 7.1 and the test-set figures above is discussed in detail in the Critical Evaluation section of this report, where it is attributed principally to the presence of attack types in the test partition that do not occur anywhere in the training data.

7.3 Confusion Matrices



The Decision Tree correctly classified 9,406 of 9,711 normal connections and 8,424 of 12,833 attack connections, producing 305 false positives and 4,409 false negatives. The Random Forest correctly classified 9,433 normal connections and 8,042 attack connections, producing 278 false positives and 4,791 false negatives. Both models therefore make comparatively few errors on normal traffic but miss a substantial share of attack traffic, a pattern examined further in the Error Analysis section below.

7.4 Errors by Attack Category

Breaking down the misclassifications produced by each model according to the underlying attack category of the test record shows that errors are heavily concentrated in two groups: attack types absent from the training data, and the r2l category.

Attack Category	DT Errors	RF Errors	Share of Total Errors (DT / RF)
Unknown / new attack types	2,302	2,646	48.8% / 52.2%
r2l (remote-to-local)	1,845	2,071	39.1% / 40.9%
normal (false positives)	305	278	6.5% / 5.5%
dos (denial-of-service)	226	40	4.8% / 0.8%
u2r (user-to-root)	31	34	0.7% / 0.7%
probe	5	~0	0.1% / ~0%
Total	4,714	5,069	100% / 100%

For both models, unseen attack types and the r2l category together account for roughly eighty-eight to ninety-three percent of all misclassifications, while the well-represented dos category, which makes up over a third of the training data, contributes a comparatively small share of errors, particularly for the Random Forest.

Error Analysis

Understanding the practical consequences of the two error types produced by an intrusion detection classifier, false positives and false negatives, is essential to judging whether a given level of accuracy is actually acceptable for deployment, since the two types of error carry very different costs in a real security context.

8.1 False Negatives: The Cost of a Missed Attack

A false negative in this context means that an actual attack connection was classified as normal and would, in a live deployment, pass through undetected. For an organisation, this is the more consequential of the two error types: an undetected intrusion can progress to data exfiltration, ransomware execution, lateral movement across internal systems, or unauthorised privilege escalation, and the resulting financial, operational, and reputational damage, along with possible regulatory consequences under data protection obligations, typically far exceeds the cost of a single unnecessary alert. The recall figures obtained in this study, 65.64 percent for the Decision Tree and 62.67 percent for the Random Forest, indicate that roughly one in three attack connections in the test set would not be flagged by either model, which is a serious limitation if these classifiers were deployed as the sole line of automated defence.

8.2 False Positives: The Cost of a False Alarm

A false positive means that a normal connection was classified as an attack, triggering an alert or a blocking action that was not actually warranted. The direct cost of a false positive is lower than that of a missed attack, but it is not negligible: security analysts who are repeatedly presented with alerts that turn out to be benign experience alert fatigue, which can lead them to deprioritise or overlook a genuine alert buried among false ones, and automated blocking based on a false positive can disrupt legitimate business activity or deny access to authorised users. Both models in this study kept false positives low, at 305 for the Decision Tree and 278 for the Random Forest out of 9,711 normal connections, corresponding to false positive rates of roughly 3.1 and 2.9 percent respectively.

8.3 The Precision–Recall Trade-off in This Context

The combination of high precision and comparatively low recall observed for both models indicates a conservative detection posture: when either model raises an alert, it is very likely correct, but a substantial share of genuine attacks slip through undetected. Whether this trade-off is appropriate depends on the deployment context. In a security operations setting where analyst time is heavily constrained and false alarms cannot be tolerated, a high-precision, lower-recall model may be preferred despite the cost of missed detections. In most intrusion detection deployments, however, the asymmetric cost of a successful intrusion relative to a false alarm means that recall is usually prioritised, since a single missed attack can cause damage that dwarfs the cumulative cost of many false alerts. Judged against this more common priority, the recall levels obtained here would generally be considered insufficient for production use without further adjustment, such as threshold tuning, cost-sensitive training, or oversampling of the minority attack categories that are currently under-detected.

8.4 Concentration of Errors in r2l and Unseen Attack Types

The error breakdown in Section 7.4 shows that the r2l category and previously unseen attack types dominate the error counts for both models, together accounting for the large majority of misclassifications. This concentration has two distinct explanations that carry different implications. The high error rate on r2l reflects the severe class imbalance documented in the Exploratory Data Analysis section, where r2l constitutes well under one percent of the training data, giving both classifiers very little material from which to learn the characteristics of this attack category. The high error rate on unseen attack types reflects a fundamentally different and more difficult challenge: no supervised classifier can correctly recognise a pattern of behaviour it has never encountered during training, which mirrors the real-world problem of zero-day attacks in operational cybersecurity, where genuinely novel attack techniques by definition fall outside the scope of any model trained exclusively on historical data. These two error sources call for different remedies, the first pointing towards better handling of minority classes during training, and the second pointing towards the need for complementary detection strategies, such as anomaly-based or

unsupervised methods, capable of flagging behaviour that does not resemble any previously observed class rather than attempting to classify it correctly.

Conclusions

9.1 Key Findings

This project reproduced a tree-based approach to network intrusion detection on NSL-KDD, informed by the feature elimination methodology of Nkiama, Said, and Saidu (2016) and the accompanying GitHub implementation. Both the Decision Tree and Random Forest classifiers achieved near-perfect cross-validation accuracy on the training data, yet fell to below 80 percent accuracy on the independent test partition, with the gap traced directly to seventeen attack types comprising 3,750 records that are entirely absent from training. Both models displayed a conservative detection posture, with precision above 96 percent but recall below 66 percent, and error analysis confirmed that misclassifications are concentrated overwhelmingly in the r2l category and in previously unseen attack types, rather than being spread evenly across all forms of attack.

9.2 Strengths and Weaknesses of the Original Approach

The feature elimination methodology proposed in the reference paper has clear strengths: it offers a systematic, classifier-guided procedure for reducing dimensionality, it is grounded in a defensible mathematical rationale linking feature importance to discriminative value, and the correlation-based redundancy identified independently in this project, such as the near-perfect relationship between num_root and num_compromised, supports the general premise that a meaningful portion of the NSL-KDD feature set is redundant and can be safely reduced. The principal weakness identified through this project's own experimentation is methodological rather than conceptual: without explicit confirmation that a feature selection or classification method has been evaluated against the full NSL-KDD test partition, including its unseen attack types, a reported accuracy figure risks reflecting performance only within the closed set of attack behaviours seen during training, which is a materially different and easier problem than detecting genuinely novel intrusions.

9.3 Suggestions for Future Improvement

Several directions could improve on the results obtained in this project and address the limitations identified above. Cost-sensitive learning or the use of class weights could push both classifiers towards higher recall at an acceptable cost in precision, directly targeting the false-negative-heavy error profile observed here. Oversampling techniques applied to the r2l and u2r categories during training could give the models more material from which to learn these rare but consequential attack patterns. Systematically reporting both training/cross-validation accuracy and independent test-set accuracy, with explicit attention to records representing previously unseen attack types, would make future feature-selection studies on NSL-KDD considerably easier to interpret and compare. Finally, complementing the supervised classifiers used here with an anomaly-detection component capable of flagging traffic that does not resemble any known class, rather than forcing a classification into one of the existing categories, would directly address the zero-day-like challenge posed by the seventeen unseen attack types identified in this study.

Summing It Up

This project set out to reproduce and critically evaluate a machine learning approach to network intrusion detection grounded in the NSL-KDD dataset, the feature elimination methodology of Nkiama, Said, and Saidu (2016), and the corresponding GitHub implementation by Cynthia Koopman. A Decision Tree and a Random Forest were trained on a reduced and cleaned version of the NSL-KDD feature set, achieving strong precision but only moderate recall on the test partition, with an accuracy of 79.09 percent and 77.52 percent respectively, well below the near-perfect cross-validation accuracy observed during training.

The central conclusion of this report is that this train-test gap, driven by seventeen attack types present only in the test data, demonstrates why high training or cross-validation accuracy alone cannot be treated as sufficient evidence of a strong intrusion detection method, whether in this project or in the reference literature it draws on. Feature engineering choices grounded in correlation analysis were shown to be both mathematically and operationally justified, the exploratory analysis clarified why rare attack categories such as r2l and u2r are so difficult to detect, and the error analysis highlighted a precision-recall trade-off that would need to be addressed before any such model could be considered suitable for real-world deployment. Overall, the exercise reinforces that genuine progress in intrusion detection research depends as much on rigorous, generalisation-aware evaluation as it does on the sophistication of the feature selection or classification technique employed.

References

Nkiama, H., Said, S. Z. M., & Saidu, M. (2016). A subset feature elimination mechanism for intrusion detection system. International Journal of Advanced Computer Science and Applications, 7(4), 148–157. https://doi.org/10.14569/IJACSA.2016.070419

Koopman, C. (n.d.). Network-Intrusion-Detection [Computer software repository]. GitHub. https://github.com/CynthiaKoopman/Network-Intrusion-Detection

Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications. NSL-KDD dataset distributed by the Canadian Institute for Cybersecurity, University of New Brunswick. https://www.unb.ca/cic/datasets/nsl.html